AIDA vs Claude Code Agent Teams (`CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS`)

AIDA positioning · best-effort comparison — live source: github.com/joemooney/aida

rendered 2026-07-10

Last updated: 2026-07-09 — Claude Code's Agent Teams surface (the experimental `CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS` flag, Claude Code >=2.1.32 — native inter-agent mailbox, a shared task-list with self-claim + file-locking + auto-unblocking dependencies, a plan-approval gate, and quality-gate hooks) is new and moving fast. Re-verify against the current Claude Code docs before treating capability claims here as authoritative; if the surface drifts in a way that changes the layer story, update this doc. Companion to vs-claude-code-subagents.md and vs-claude-code-workflows.md.

TL;DR: A Claude Code **Agent Team** is a *within-session* coordination primitive — several Claude agents that share a local-JSON mailbox and a task-list, self-claim work, lock files against each other, auto-unblock dependent tasks, and gate on plan approval, all inside one running session that ends when you close it. **AIDA** is a *cross-session* substrate + lifecycle — a durable git-canonical requirement graph with stable IDs, typed relations, code→spec traces, an escalation ladder, and an autonomous drain that ships real PRs and tracks spec state over time, across agents **and vendors**. This is the most-overlapping provider primitive yet — it lands squarely on AIDA's *coordination* layer, not just its agent layer. The honest answer is still **no, not the same thing** — but the rhyme is now loud enough that naming the boundary precisely is what makes it stick. They **compose**: AIDA seeds and harvests an Agent Team; AIDA is the layer above.

The worry that prompts this doc is the sharpest one yet. Earlier Claude Code primitives (subagents, Workflows) overlapped AIDA's *agent* and *orchestration* surfaces. Agent Teams overlaps the **coordination** surface — a shared task-list with self-claim, file-locking, and dependency auto-unblocking is, feature-for-feature, the closest thing Anthropic has shipped to AIDA's queue + lease + blocked-by machinery. A reasonable reader sees "shared task list, claim a task, dependencies unblock automatically, agents message each other" and asks *am I getting AIDA's coordination layer for free, built in?* The honest answer is **no** — but the reasons are specific, and they're about *where the state lives and how long it lives*, not about whose feature list is longer.

What each one actually is

	Claude Code Agent Teams	AIDA (graph + queue + drain)
What it is	Several Claude agents in one session sharing a mailbox + task-list	A persistent requirement graph + queue + lifecycle that any vendor's agent drives
Unit of coordination	A **task** on the shared list, claimed for the duration of the session	A **spec** with a stable ID, moved through its development lifecycle
Where the state lives	Local JSON, scoped to the running session	Git-canonical: orphan `aida-store` branch, YAML per spec, history, traces in source
Lifetime	**Session-ephemeral** — the mailbox, the task-list, the claims, the locks all die when the session ends; there is no session resumption	**Cross-session** — the spec, queue entry, lease, status, and trace comments survive every session ending
Mutual exclusion	**File-locking** between teammates inside the session	**Leases** on specs/worktrees that survive the process and are queryable by the next session
Dependencies	**Auto-unblocking** within the task-list — a finished task releases its dependents	**Typed `blocked-by` / `blocks` relations** in the graph, queryable transitively (`aida graph --blocked-by`), durable across sessions
Approval	A **plan-approval gate** before the team executes	An implementer → advisor → human **escalation ladder**; reviewer verdicts; punt handshake
Quality gates	**Hooks** that fire on team events	Commit-time trace enforcement, CI phase, reviewer phase (with a structural self-merge guard the implementer can't bypass), auto-bump-on-merge
Identity	None — agents share a task-list, not a requirement graph	Stable **SPEC-IDs** that survive renames, merges, and vendor switches
Vendor	**Claude Code only**	Cross-vendor — Claude, Codex, Antigravity all drive one git-canonical substrate
The shape	**A coordinated set of agents you run**	**A durable substrate + a lifecycle that runs on it**

An Agent Team is a *coordinated crew that works a shared list and disbands*. AIDA is a *system that persists what the crew did and why*. Not the same kind of thing.

The separating test

Two questions decide which layer you're in:

1. **Does the coordination outlive the session, anchored to a durable identity?**
   - Agent Team → **no.** The mailbox, task-list, claims, and locks live in session-local JSON; close the session and the coordination state is gone. There is no resumption: tomorrow's session does not inherit today's task-list.
   - AIDA → **yes.** The spec is in git with a stable ID, a status, typed relations, and trace comments in the code — the queue entry and lease are queryable next week by a *different agent on a different vendor*.
2. **What is the repeatable, durable thing?**
   - Agent Team → **the task-list within a run.** It coordinates *this* session's agents over *this* session's tasks, then ends.
   - AIDA → **the substrate + the lifecycle semantics.** The graph, the stable IDs, and "what Done vs Completed means" persist; *which* agents coordinated on a given drain is incidental and not the artifact.

If your answer to "where does the coordination state go when the session ends?" is *"into a graph other agents query later,"* you're in AIDA's layer. If it's *"it's gone — it was scoped to the run,"* you're in an Agent Team.

Overlap — said honestly

This is the closest overlap of any Claude Code primitive, so it earns a frank inventory. Agent Teams genuinely ships things that rhyme with AIDA's coordination layer:

- **A shared task-list with self-claim** rhymes with AIDA's **queue** (`aida queue list / next / work`).
- **File-locking between teammates** rhymes with AIDA's **leases** (`aida session leases`).
- **Auto-unblocking dependencies** rhymes with AIDA's **typed `blocked-by` relations** and the resilient drain's dependent-skipping.
- **A plan-approval gate** rhymes with AIDA's **advisor sign-off** before approved work enters the ready set.
- **Quality-gate hooks** rhyme with AIDA's **commit-time trace enforcement + reviewer phase**.
- **An inter-agent mailbox** rhymes with AIDA's **briefs + MCP coordination surface** (`list_briefs`, `send_message`).

The honest read: **Agent Teams' *coordination mechanism* and AIDA's queue/lease layer are the same category of thing — within a session.** What differs is everything *around* the mechanism — the durability, the identity, the cross-session lifecycle, the cross-vendor reach. The overlap is real and it is on the coordination surface, not a surface AIDA can wave away.

Why AIDA's coordination can't just BE an Agent Team

The natural follow-up: *"if Agent Teams ships a shared task-list with claims, locks, and dependencies, why doesn't AIDA just use it for coordination?"* The answer is that the things AIDA's coordination is made of don't exist at the Agent-Team layer:

- **No persistence.** The task-list and mailbox are session-local JSON; they die when the session ends. You cannot build a queue that survives across days out of a list that vanishes at session close.
- **No session resumption.** A fresh session does not inherit a prior team's task-list. AIDA's queue, leases, and spec statuses are *designed* to be picked up cold by the next session — that is the whole point.
- **No graph anchoring.** A task on the shared list is a string of work, not a node in a typed graph. "Block STORY-489 on STORY-545, and show me the transitive closure next month" needs stable IDs and typed relations that a within-session list has no representation for.
- **No cross-vendor reach.** Agent Teams coordinates *Claude* agents; a Claude-only primitive doesn't reach a Codex or Antigravity session. AIDA's queue routes the same spec across vendors against one shared git-canonical store. Anthropic *could* extend Agent Teams cross-vendor — but that cuts against the incentive to keep coordination inside the Claude ecosystem, which is why the gap is likely to persist, not because it's technically unbridgeable.
- **No durable lifecycle.** The team gates on plan approval and runs; it does not carry a spec from `Draft → … → Completed` with an auto-bump-on-merge that a *different* session, weeks later, can trust. AIDA's lifecycle semantics are the durable thing; the team's gate is a within-run checkpoint.

AIDA's defensible niche **is** the persistent, vendor-neutral graph + the cross-session lifecycle. You cannot build it out of session-scoped coordination — wrong layer. The queue, the leases, the typed relations, the trace comments, the auto-bump, the MCP exposure across vendors — none of these are things a within-session team can carry.

They compose

The complementary picture, and it runs both ways: **AIDA seeds an Agent Team, and harvests it back.**

1. AIDA → Agent Team (AIDA seeds the team). AIDA's graph is the natural *input* to a team. Before launching a team, ask AIDA (over MCP or CLI) what's ready and how it's related:

- `aida queue list` → the ready specs become the team's task-list.
- `aida graph <ID> --blocked-by` → the typed dependency edges seed the team's auto-unblocking order, so the team's within-session dependency graph is grounded in the durable one rather than re-derived from prose.
- `show_requirement <ID>` (MCP) → each task carries its spec's acceptance criteria into the team, so teammates work against what the spec actually requires, not a generic restatement.

The team then coordinates *this run's* execution — claims, locks, mailbox — against a task-list that AIDA populated with real, related, acceptance-bearing specs.

2. Agent Team → AIDA (AIDA harvests the team). When the team finishes, the work has to land *somewhere durable* or the knowledge leaks away the moment the session closes. AIDA is that destination:

- Each teammate commits with a `(SPEC-ID)` trailer → the merge auto-bumps the spec `Done → Completed` in the graph.
- Trace comments in the shipped code → the code→spec edges survive the team's disbanding.
- Status flips, lease releases, and history rows → land in git-canonical YAML, queryable next week by a different session on a different vendor.

The team is **a coordinated crew AIDA dispatches and reabsorbs**, not a replacement for the substrate. Agent Teams makes AIDA's within-session execution *better* — native mailbox, native locking, native dependency-unblocking, none of which AIDA wants to hand-roll. AIDA is the layer above: it supplies the team's durable input and captures its durable output.

The right mental model is a two-level stack: **Agent Teams** are the within-session coordination layer; **AIDA** is the cross-session substrate + lifecycle layer. Both are useful. Neither replaces the other.

Why the gap persists — it's incentive, not capability

It would be a mistake to read this doc as *"Anthropic hasn't gotten around to durability yet."* The honest, longer-lasting read is that **the gap is structural, and it rests on incentive, not capability.**

Anthropic is a single-vendor business by design: Claude Code coordinates *Claude* agents, and a richer Claude-Teams coordination layer makes the Claude product stickier. A *vendor-neutral, git-canonical* substrate that lets a Codex or Antigravity session pick up the same spec is not a feature Anthropic is incented to build — it would dilute exactly the lock-in that a single-vendor coordination layer is meant to create. The session-scoped, local-JSON, Claude-only shape of Agent Teams is not an oversight on the way to durability; it is the shape that aligns with the business model.

That is why this is the **load-bearing** claim, and why it ages better than any capability claim: Anthropic could clearly *build* durable cross-vendor coordination — the engineering is not the obstacle. The obstacle is that doing so would work against its own incentive. AIDA's neutrality is not a head-start Anthropic will erase next release; it's a position Anthropic is structurally disinclined to take. Capability claims about a fast-moving competitor rot in weeks; the incentive argument is the one that holds.

The corollary for AIDA's own discipline: **don't reinvent the within-session coordination Agent Teams now ships natively.** If AIDA finds itself building "let agents message each other and lock files within one running session," stop — that's Agent Teams. AIDA's niche is strictly the durable, cross-vendor graph + lifecycle layer; below that line, defer to the Claude Code primitive and shell out to it.

Honest scope statement

What Agent Teams do that AIDA doesn't: native within-session multi-agent coordination — a shared mailbox, self-claim from a task-list, file-locking between teammates, auto-unblocking dependencies, a plan-approval gate, and quality-gate hooks — all built into Claude Code, no infrastructure to stand up. If you want several Claude agents to divide and coordinate the work *of one session* without stepping on each other, that's an Agent Team, not an AIDA construct.

What AIDA does that Agent Teams don't: a persistent requirement graph; stable IDs across renames and vendors; typed relations queryable transitively; code→spec traces enforced at commit; a cross-session lifecycle with auto-bump-on-merge; a human-escalation ladder; and a git-canonical substrate any vendor's agent can read. If you need to know *what exists and why* six months from now — or to coordinate a *Codex* session against the same work — that's AIDA, not a team that disbanded when the session closed.

A serious multi-agent project uses **both**: AIDA for the spec / lifecycle / traceability / cross-vendor substrate, Agent Teams for the within-session coordination of the Claude agents that drive it. AIDA seeds the team from the graph and harvests the team's output back into it.

See also

- vs-claude-code-subagents.md — the agent-layer distinction (subagents are the *callable*; AIDA roles are the *position*).
- vs-claude-code-workflows.md — the orchestration-layer distinction (a Workflow is a *scriptable fan-out*; AIDA is a *durable substrate + lifecycle*).
- Claude Code Agent Teams docs — the vendor's authoritative surface; re-verify capability claims here against it.
- docs/competitive-analysis/2026-06-09-weekly-scan.md — the weekly scan (Lane B, finding B1) that prompted this doc.
- docs/competitive-analysis/2026-05-31-round2-moat-gaps-moves.md — the current moat / commoditization synthesis.